

Remodelagem do fluxo de desenvolvimento com IA

Análise do método de Matt Pocock e proposta de adaptação para o Orquestrador-Maestro

Resumo executivo

O método de Matt Pocock oferece uma disciplina forte para transformar ideias vagas em mudanças implementáveis, verificáveis e revisáveis. A melhor adaptação para o Orquestrador-Maestro é combinar fases controladas pelo maestro com slices verticais executáveis, mantendo gates humanos entre decisões importantes.

Documento de recomendação
19 de julho de 2026

1. O que foi pesquisado

A pesquisa foi baseada no repositório público de skills do Matt Pocock, nos materiais do AI Hero e em resumos do workshop sobre fluxo de desenvolvimento com agentes. O núcleo do método é composto por alinhamento, especificação, decomposição, implementação incremental e revisão independente.

O fluxo clássico pode ser resumido como:

grill-with-docs → to-prd → to-issues → tdd/implement → review

O método também inclui ferramentas complementares para prototipagem, diagnóstico, handoff, análise arquitetural e melhoria contínua. A mensagem recebida pelo usuário mistura esse fluxo com uma adaptação em fases, mais adequada para avaliação humana progressiva.

2. Principais ideias aproveitáveis

Ideia	Valor	Adaptação recomendada
Alinhamento rigoroso	Reduz interpretações erradas antes do código.	Criar uma entrevista limitada, baseada no código e na documentação.
PRD como destino	Mantém problema, decisões e critérios de aceite explícitos.	Usar DEV/SPECS/ como fonte de verdade local.
Slices verticais	Entrega feedback integrado cedo.	Usar slices dentro das fases, atravessando banco, domínio, API, UI e testes.
Execução AFK	Automatiza tarefas bem especificadas.	Permitir apenas em itens classificados como AFK e com limites de execução.
Revisão em contexto novo	Evita que o próprio agente valide seus pressupostos.	Separar implementação, revisão técnica e aprovação humana.

3. Modelo recomendado para o Orquestrador-Maestro

A proposta não é substituir as capacidades existentes, mas conectá-las em um fluxo único e explícito:

Ideia → Contexto → PRD → Fases → Slices → Implementação → Verificação → Revisão → Retrospectiva

3.1 Fases e slices

Fases organizam arquitetura, dependências, riscos e decisões. Slices verticais organizam a execução. Uma fase pode conter diversos slices, e cada slice precisa produzir algo demonstrável ou verificável por conta própria.

Exemplo de decomposição:

Fase	Slices possíveis	Gate humano
Fundamentos	modelo mínimo; migração; serviço; API mínima; tela integrada	validar domínio, dados e contratos
Regras de negócio	fluxo principal; permissões; erros; estados de exceção	validar regras e casos-limite
Qualidade	refatoração; segurança; UX; acessibilidade; observabilidade	aprovar risco residual e comportamento final

3.2 Contrato de cada slice

Cada slice deve declarar tipo **HITL** (human-in-the-loop) ou **AFK** (away-from-keyboard), dependências, camadas afetadas, critérios de aceite, testes e comando de verificação. Testes não devem virar uma etapa horizontal isolada; devem acompanhar a entrega do comportamento.

3.3 Impacto arquitetural explícito

O plano deve distinguir: **deve ser alterado**, **provavelmente será alterado**, **pode ser impactado** e **não deve ser tocado**. Essa distinção reduz mudanças acidentais e torna a revisão mais objetiva.

4. Skills e artefatos a agregar

grill-with-docs

Entrevista baseada no repositório; atualiza CONTEXT.md e ADRs; não implementa automaticamente.

to-prd

Converte o alinhamento em especificação com escopo, fora de escopo, regras, decisões, riscos, critérios de aceite e testes.

prd-to-phases

Transforma a especificação em fases, dependências, impacto esperado e gates humanos.

phase-to-slices

Gera slices verticais locais com tipo HITL/AFK, camadas, critérios de aceite e verificação.

improve-codebase-architecture

Produz relatório visual de dependências, módulos rasos, riscos de mudança e oportunidades de aprofundamento.

prototype

Valida interface, estados e decisões de produto antes de comprometer a implementação.

review independente

Avalia fidelidade ao PRD, arquitetura, testes, segurança, UX e dívida técnica em contexto novo.

retrospectiva

Converte erros recorrentes e decisões difíceis em documentação, checklists e melhorias de skills.

Estrutura sugerida:

DEV/CONTEXT.md — vocabulário e contexto do domínio

DEV/SPECS/ — PRDs e especificações ativas

DEV/PHASES/ — planos aprovados por fase

DEV/SLICES/ — unidades executáveis

DEV/ADR/ — decisões arquiteturais

DEV/REPORTS/ — relatórios visuais e revisões

5. Guardrails essenciais

A adoção precisa preservar o controle do maestro e evitar que o processo se transforme em automação sem supervisão.

Limitar a entrevista

Usar um orçamento de perguntas, por exemplo 12 a 20 perguntas críticas, com encerramento quando a confiança estiver suficiente. A entrevista deve explorar o código quando a resposta puder ser descoberta localmente.

Limitar o modo AFK

Somente slices pequenos, com critérios de aceite objetivos, testes executáveis, escopo fechado e rollback simples. Nunca delegar decisões de arquitetura, segurança ou mudança de dados sem aprovação.

Separar papéis

O agente que implementa não deve ser a única fonte da revisão. A revisão deve ocorrer em contexto novo e a aprovação final de decisões importantes permanece humana.

Evitar importação indiscriminada

O valor está na composição disciplinada, não na quantidade de skills. O repositório já possui planner, architect, reviewer, Ralph, team, handoff e documentação DEV; a prioridade é roteá-los e conectá-los.

6. Roteamento proposto

Situação	Fluxo
Ideia vaga	grill-with-docs → to-prd
Feature grande	to-prd → prd-to-phases → phase-to-slices
Fase aprovada	tdd/implement → testes → review
Código confuso	zoom-out → improve-codebase-architecture
Mudança visual	prototype → design review → implementation
Bug ou regressão	diagnose → reprodução → correção → teste de regressão

7. Roadmap de implementação

A remodelagem pode ser feita incrementalmente, sem interromper o fluxo atual.

Prioridade 1 — Integração documental

Definir os formatos de CONTEXT, PRD, fase e slice em DEV. Atualizar o roteador para reconhecer a sequência e manter gates humanos.

Prioridade 2 — Fases e slices

Criar as skills prd-to-phases e phase-to-slices, reutilizando planner, architect, team e Ralph quando adequado.

Prioridade 3 — Contexto visual

Adicionar relatório visual de arquitetura e impacto de mudanças; incluir protótipo para tarefas de interface.

Prioridade 4 — Qualidade e aprendizagem

Adicionar revisão independente, retrospectiva e métricas simples: tempo por fase, retrabalho, falhas encontradas na revisão e consumo de tokens.

8. Conclusão

O fluxo de Matt Pocock é uma boa referência porque transforma a interação com a IA em um processo de engenharia: primeiro alinhamento, depois especificação, decomposição, execução incremental e verificação. A adaptação em fases do Orquestrador-Maestro é ainda mais adequada ao objetivo de economizar tokens e permitir avaliação contínua.

A recomendação é adotar um fluxo híbrido: **fases para governança e slices verticais para entrega**. Com isso, o sistema ganha previsibilidade, melhor feedback, menos contexto desperdiçado e maior clareza sobre o impacto de cada mudança.

Referências consultadas

1. [GitHub — mattpocock/skills](#)
2. [AI Hero — AI Engineering Posts](#)
3. [TalksIntel — Full Walkthrough: Workflow for AI Coding](#)
4. [GitHub Issue #132 — limite de entrevistas](#)
5. [GitHub Issue #134 — proteção contra implementação automática](#)